

CAPSTONE PROJECT

USECASE 4 – MySQL Reporting & Integration

Name : Mamidoju.Vishal

Email : vishalmamidoju5@gmail.com

Objective: Store, query, and present analytical results from MySQL.

Tasks

1. Design & Populate Summary Tables:

- Create tables and load the data (should be done thru Python MySQL Connector).

Code:

```
# Connect to SQLite
try:
    connection = sqlite3.connect(DATABASE_FILE)
    cursor = connection.cursor()

    print("\nSuccessfully connected to SQLite.")

except Exception as error:
    print("\nERROR CONNECTING TO SQLITE:")
    print(error)
    raise SystemExit

try:
    df = pd.read_csv(CSV_FILE)

except Exception as error:
    print("\nERROR READING CSV FILE:")
    print(error)

    connection.close()
    raise SystemExit
```

```

# Normalize column names
df.columns = df.columns.str.strip().str.lower()

print("\nCrime dataset loaded successfully.")
print("Total records:", len(df))
print("Dataset columns:")
print(df.columns.tolist())

# Check required columns
missing_columns = [
    column
    for column in required_columns
    if column not in df.columns
]

if missing_columns:
    print("\nERROR: Required columns are missing!")
    print("Missing columns:", missing_columns)

    connection.close()
    raise SystemExit

```

Output:

```

Successfully connected to SQLite.

Crime dataset loaded successfully.
Total records: 2000
Dataset columns:
['id', 'case_number', 'date', 'block', 'iucr_code', 'primary_type', 'description', 'location_desc', 'arrest', 'domestic', 'beat_num', 'district_code', 'ward_no', 'community_code', 'fbi_code', 'x_coordinate', 'y_coordinate', 'year', 'date_of_update', 'latitude', 'longitude', 'location']

Data preparation completed successfully.
|
Main table chicago_crimes created successfully.

2000 records inserted successfully.

Summary tables created successfully.

```

2. MySQL Queries:

- Write SQL to compute:
 - Crime count per year.
 - Top 5 crimes and their percentages.
 - Arrest count per year.

Code:

```
# Crime count per year
crime_yearly_query = """
SELECT
    crime_year,
    COUNT(*) AS crime_count
FROM chicago_crimes
WHERE crime_year IS NOT NULL
GROUP BY crime_year
ORDER BY crime_year
"""

cursor.execute(crime_yearly_query)
crime_per_year = cursor.fetchall()
```

Output:

```
===== CRIME COUNT PER YEAR =====
(2015, 227)
(2016, 242)
(2017, 192)
(2018, 213)
(2019, 236)
(2020, 224)
(2021, 229)
(2022, 221)
(2023, 216)
```

Code:

```
--  
top_5_crimes_query = """  
SELECT  
    primary_type,  
    COUNT(*) AS crime_count,  
    ROUND(  
        COUNT(*) * 100.0 /  
        (SELECT COUNT(*) FROM chicago_crimes),  
        2  
    ) AS crime_percentage  
FROM chicago_crimes  
GROUP BY primary_type  
ORDER BY crime_count DESC  
LIMIT 5  
""">  
  
cursor.execute(top_5_crimes_query)  
top_5_crimes = cursor.fetchall()  
  
print("\n===== TOP 5 CRIME TYPES =====")  
  
for row in top_5_crimes:  
    print(row)
```

Output:

```
===== TOP 5 CRIME TYPES =====  
( 'THEFT', 416, 20.8)  
( 'BATTERY', 328, 16.4)  
( 'ASSAULT', 195, 9.75)  
( 'NARCOTICS', 178, 8.9)  
( 'BURGLARY', 177, 8.85)
```

Code:

```
# Arrest count per year
arrest_yearly_query = """
SELECT
    crime_year,
    SUM(arrest) AS arrest_count
FROM chicago_crimes
WHERE crime_year IS NOT NULL
GROUP BY crime_year
ORDER BY crime_year
"""

cursor.execute(arrest_yearly_query)
arrest_per_year = cursor.fetchall()

print("\n===== ARREST COUNT PER YEAR =====")

for row in arrest_per_year:
    print(row)
```

Output:

```
===== ARREST COUNT PER YEAR =====
(2015, 63)
(2016, 62)
(2017, 56)
(2018, 72)
(2019, 75)
(2020, 68)
(2021, 68)
(2022, 72)
(2023, 60)
```

3. Database Stored Views:

- Create views in MySQL:
 - vw_crime_yearly
 - vw_crime_by_category

Code:

```
cursor.execute(
    "DROP VIEW IF EXISTS vw_crime_yearly"
)

cursor.execute("""
CREATE VIEW vw_crime_yearly AS
SELECT
    crime_year,
    COUNT(*) AS crime_count,
    SUM(arrest) AS arrest_count
FROM chicago_crimes
WHERE crime_year IS NOT NULL
GROUP BY crime_year
""")

cursor.execute(
    "DROP VIEW IF EXISTS vw_crime_by_category"
)

cursor.execute("""
CREATE VIEW vw_crime_by_category AS
SELECT
    primary_type,
    COUNT(*) AS crime_count,|
    ROUND(
        COUNT(*) * 100.0 /
        (SELECT COUNT(*) FROM chicago_crimes),
        2
    ) AS crime_percentage
FROM chicago_crimes
GROUP BY primary_type
""")

connection.commit()

print("\nDatabase views created successfully.")
```

Output:

```
===== YEARLY CRIME DATA =====
  crime_year  crime_count  arrest_count
0        2015          227           63
1        2016          242           62
2        2017          192           56
3        2018          213           72
4        2019          236           75
5        2020          224           68
6        2021          229           68
7        2022          221           72
8        2023          216           60

===== TOP CRIME CATEGORY DATA =====
  primary_type  crime_count  crime_percentage
0          THEFT          416           20.80
1        BATTERY          328           16.40
2        ASSAULT          195           9.75
3      NARCOTICS          178           8.90
4        BURGLARY          177           8.85
5        ROBBERY          165           8.25
6  MOTOR VEHICLE THEFT          118           5.90
7        VANDALISM          115           5.75
8  PUBLIC PEACE VIOLATION           76           3.80
9    DECEPTIVE PRACTICE           68           3.40
```

4. Pandas Integration:

- Read these views into Pandas DataFrames for further analysis:

```
pd.read_sql("SELECT * FROM vw_crime_yearly", conn)
```

Code:

```
yearly_df = pd.read_sql("""
SELECT *
FROM vw_crime_yearly
ORDER BY crime_year
""", connection)

category_df = pd.read_sql("""
SELECT *
FROM vw_crime_by_category
ORDER BY crime_count DESC
""", connection)

print("\n===== YEARLY CRIME DATA =====")
print(yearly_df)

print("\n===== TOP CRIME CATEGORY DATA =====")
print(category_df.head(10))
```

Output:

```
SQLite Database:
C:\Users\visha\Downloads\capstone project\capstone project\chicago_crime_usecase
4.db

Main Table:
chicago_crimes

Summary Tables:
crime_yearly_summary
crime_category_summary

Database Views:
vw_crime_yearly
vw_crime_by_category

Graphs Saved:
crime_yearly.png
arrest_yearly.png
top_categories.png

Graph folder:
C:\Users\visha\Downloads\capstone project\capstone project\static\charts\usecase
4
```

5. Visualization from MySQL Data:

- Plot SQL extracted data with Matplotlib and Seaborn.

Code:

```
# Graph 1: Crime count per year
plt.figure(figsize=(10, 5))

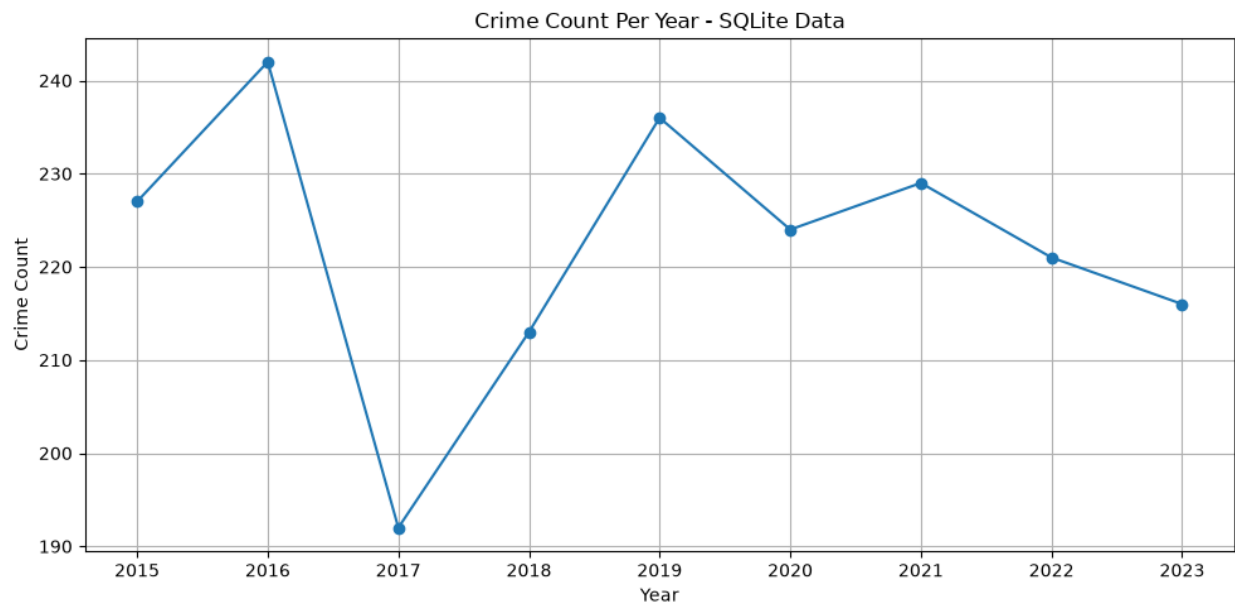
plt.plot(
    yearly_df["crime_year"],
    yearly_df["crime_count"],
    marker="o"
)

plt.title("Crime Count Per Year - SQLite Data")
plt.xlabel("Year")
plt.ylabel("Crime Count")
plt.grid(True)
plt.tight_layout()

plt.savefig(
    os.path.join(
        CHART_DIR,
        "crime_yearly.png"
    ),
    bbox_inches="tight"
)

plt.close()
```

Output:



Code:

```
# Graph 2: Arrest count per year
plt.figure(figsize=(10, 5))

plt.bar(
    yearly_df["crime_year"].astype(str),
    yearly_df["arrest_count"]
)

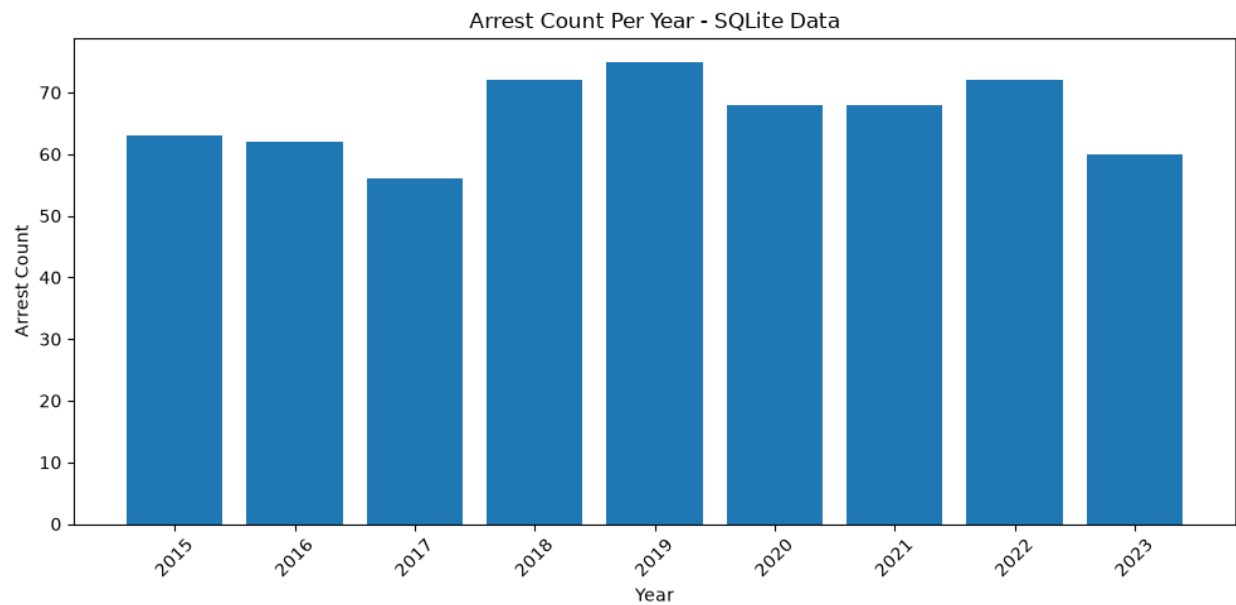
plt.title("Arrest Count Per Year - SQLite Data")
plt.xlabel("Year")
plt.ylabel("Arrest Count")

plt.xticks(rotation=45)
plt.tight_layout()

plt.savefig(
    os.path.join(
        CHART_DIR,
        "arrest_yearly.png"
    ),
    bbox_inches="tight"
)

plt.close()
```

Output:



Code:

```
# Graph 3: Top crime categories
top_categories = category_df.head(10)

plt.figure(figsize=(12, 6))

sns.barplot(
    data=top_categories,
    x="crime_count",
    y="primary_type"
)

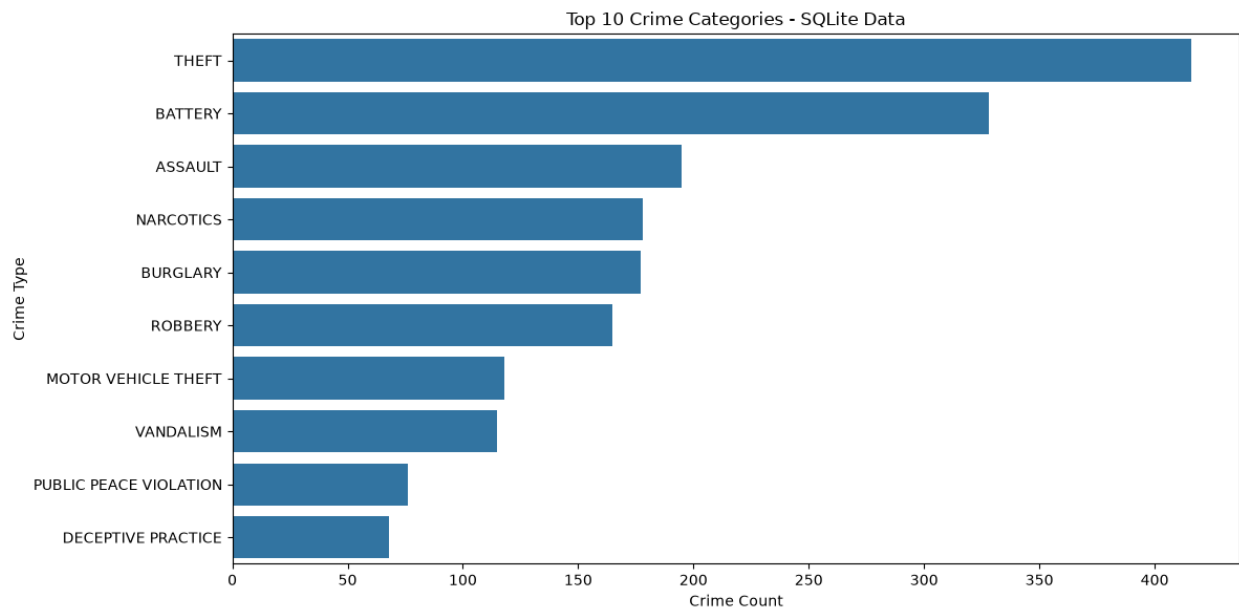
plt.title("Top 10 Crime Categories - SQLite Data")
plt.xlabel("Crime Count")
plt.ylabel("Crime Type")

plt.tight_layout()

plt.savefig(
    os.path.join(
        CHART_DIR,
        "top_categories.png"
    ),
    bbox_inches="tight"
)

plt.close()
```

Output:



Insight

- Crime data was stored and managed using **SQLite**.
- SQL queries were used to analyze yearly crime counts and arrest counts.
- Summary tables were created for faster reporting.
- Database views were created to simplify data retrieval.
- Pandas was integrated with SQLite to read and analyze query results.
- Charts were generated to visualize crime trends, arrests, and top crime categories.